IT457 / CLOUD COMPUTING

Week 2 Progress Report

CoWIN case study: high availability and failover

High-level architecture, PoC plan and team work distribution

Project theme	Government and public digital platforms
Practical focus	Two-zone cloud proof of concept (PoC)
Report stage	Week 2 design and implementation planning

Project team

Name	Student ID
Jeetrajsinh Gohil	202303017
Kaushal Nandaniya	202303036
Darshil Gandhi	202301056

Status note. Week 1 research is available in the project repository. This report presents the Week 2 design and proposed work distribution; deployment, testing, measured results and individual contributions are planned and should be updated when completed.

1. Project context and scope

CoWIN is used as a case study for examining the availability needs of a public digital platform. The Week 1 repository contains a research summary covering the problem, reported scale, architecture, outcomes and possible sources. The figures and technology choices in that summary are research claims, not experimental results from this team [1].

The course guide asks the Government and Public Platforms theme to demonstrate high availability and disaster recovery through a national-scale architecture diagram, a working component across zones, and a failover demonstration [2]. The instructor clarified that the named platform provides case-study context; a miniature version of the complete platform is not expected. Accordingly, the earlier proposed distributed slot-reservation engine is replaced by a smaller availability-focused PoC.

Problem statement. A server failure must not make a critical public-facing service permanently unavailable. The exercise investigates how a load balancer and redundant application instances in separate availability zones can limit the effect of a server failure.

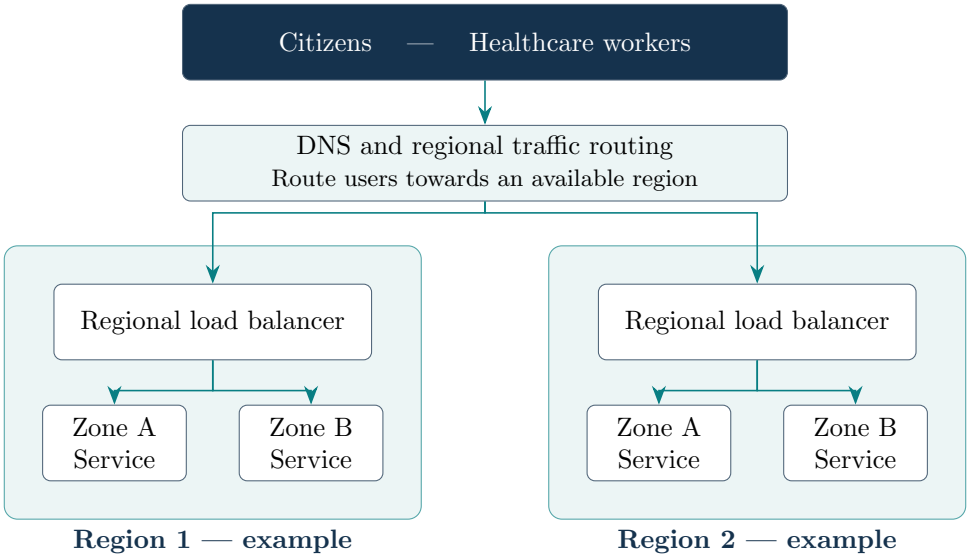
Objectives.

- Design a conceptual multi-region architecture and identify the smaller component to implement.
- Deploy a simple lookup API to two servers in different availability zones within one AWS region.
- Demonstrate and measure the impact of stopping one server while requests continue.
- Document the design, costs, evidence, limitations and each member's contribution.

Scope boundary. The PoC uses static sample data; it does not implement real booking, citizen records, a production database, inter-region failover or CoWIN's actual infrastructure.

2. Conceptual national-scale architecture

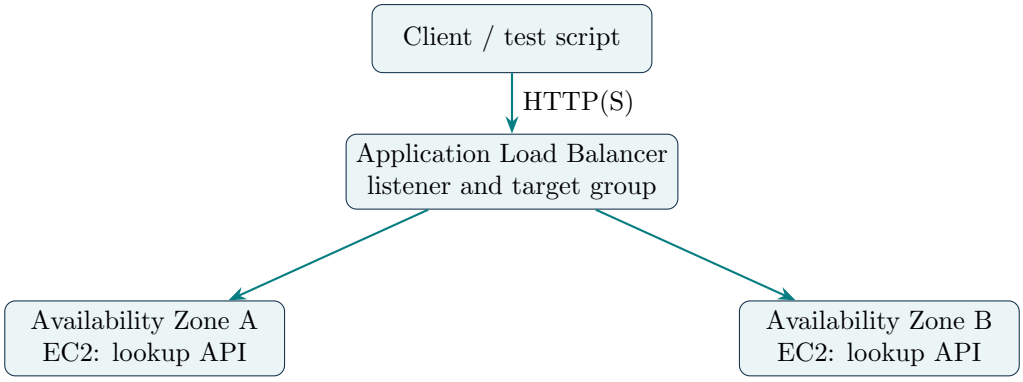
The following is a proposed design, not a verified diagram of CoWIN. Two regions are illustrative, not a required or fixed number. DNS-based traffic routing could direct requests to regional endpoints. Within each region, a load balancer would direct traffic to healthy services across zones. A production implementation would also need a carefully designed data layer, regional recovery procedures and a data consistency strategy; these are outside the PoC.



Regional routing and the production data layer are conceptual.
The PoC implements the two-zone service pattern in one region.

3. Proposed proof of concept

Scope. The team plans to deploy a simple vaccine-center lookup service on two servers in different availability zones within one AWS region. A load balancer will distribute requests between them. The service will return static sample data; building a complete vaccination platform is outside the exercise.



Purpose. This scaled-down deployment tests whether the service remains accessible when one server is stopped. The architecture illustrates availability-zone redundancy; it does not implement the multi-region design shown in Section 2.

Expected behavior. When both servers are healthy, the load balancer can route requests to either one. When one server is stopped and detected as unhealthy, requests should be directed to the remaining healthy server. Detection and recovery take time, so the test will record any interruption rather than assume that failover is instantaneous.

Final evidence required by the guide. A national-scale architecture diagram, the working two-zone component, and a video or live demonstration of a server failure and recovery.

4. Implementation and validation plan

1. Select an AWS region, instance sizes and a budget; configure a VPC and two availability zones.
2. Deploy the same small API to both instances; confirm the health endpoint works locally.
3. Configure the ALB, listener, target group, security groups and health checks. Record the actual configuration.
4. Establish a baseline by sending repeated requests through the ALB and confirming both targets are healthy.
5. Continue requests while stopping instance A. Record the failure time, target health transition and client-side results.
6. Restore instance A, verify that it becomes healthy, and save screenshots, logs and a video or live demonstration.
7. Clean up resources after evidence collection, subject to the team's submission needs.

Measurements. Record total requests, successful responses, errors, response times and timestamps. Estimate the observed disruption window from the first failed request to the first sustained successful responses, stating the request interval and observation limits. Report the measured outcome even if there is no observed failure. The experiment tests an instance failure, not the loss of an entire availability zone or a full regional disaster.

Acceptance evidence. Both targets are initially healthy; a stopped target is marked unhealthy; requests after detection reach the remaining instance; and the team provides a timestamped request log plus AWS health screenshots and a recording. Any failed requests or unexpected behavior will be reported.

Cost and operations. Before deployment, estimate the cost of EC2, the ALB, storage and any data transfer using current regional prices and expected run time. Actual charges depend on account eligibility, resource configuration and usage. Record start and stop times and remove unused resources after the demonstration.

5. Work distribution

The table assigns proposed ownership. Actual contributions, including infrastructure, research, testing and documentation, will be recorded as work proceeds; they are not limited to GitHub commits.

Member	Week 2: design and planning	Week 3: implementation and evidence
Jeetrajsinh Gohil 202303017	Problem definition, high-availability research, national-scale architecture and report coordination.	Review deployed architecture, help conduct the failover demonstration and document observations.
Kaushal Nandaniya 202303036	Review Week 1 sources; design AWS topology, resource choices and preliminary cost estimate.	Configure instances, networking, load balancer and health checks; capture infrastructure evidence.
Darshil Gandhi 202301056	Plan the simple service, failover experiment and evidence collection.	Help build the service and test it; collect logs, screenshots and demonstration evidence.

All members will review the report and presentation. The allocation can change based on actual work and availability.

6. References

[1] Kaushal Nandaniya, *CoWIN: Cloud Platform for COVID-19 Vaccination Scheduling at National Scale*, project repository and Week 1 summary. github.com/kaushalnandaniya/CoWIN.

[2] *IT457 Project: Final Deliverables & Step-by-Step Implementation Guide*, course document, Theme 5: Government and Public Platforms.

[3] Instructor clarification shared with the team, 22 October 2026: the exercise is a cloud PoC and contribution may include cloud work beyond code commits.